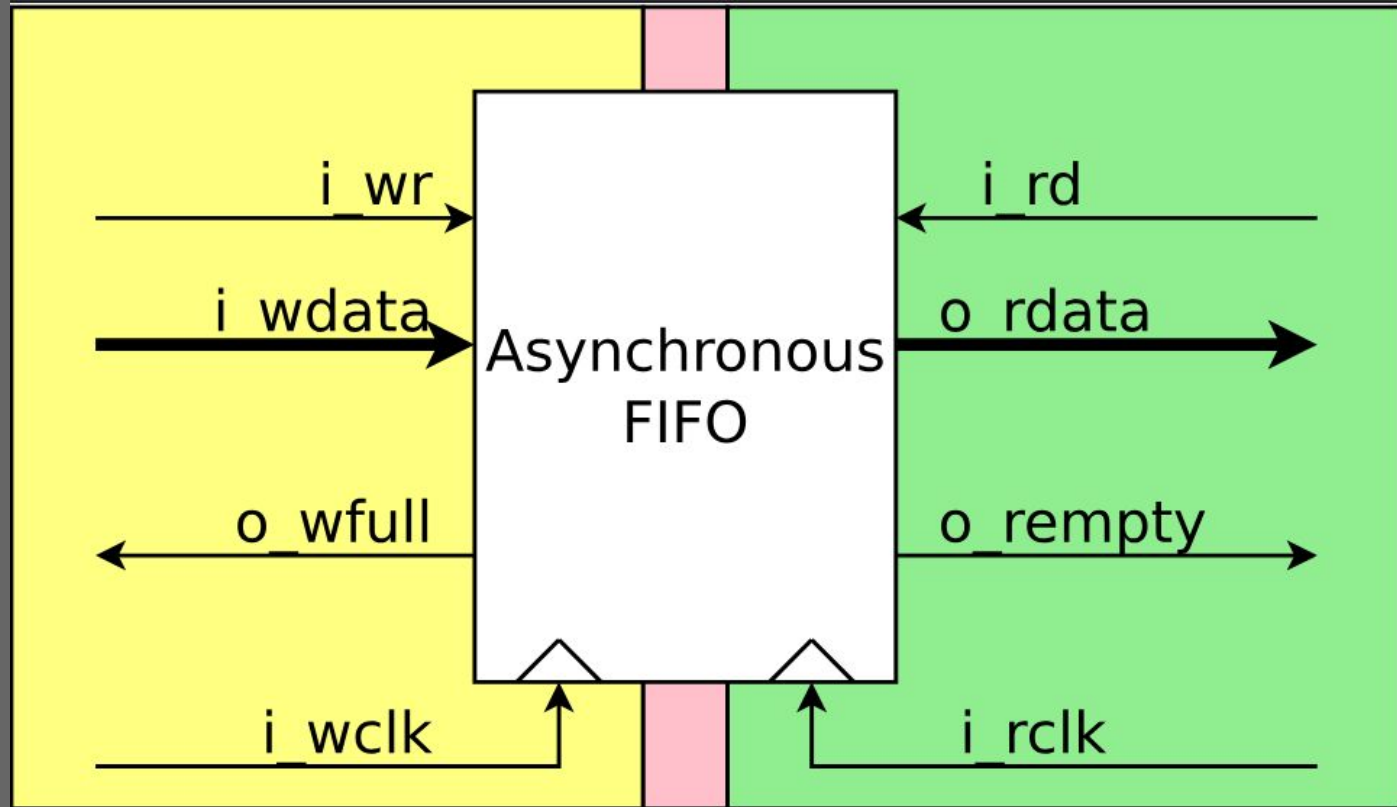


CPE 470 - Async Fifos

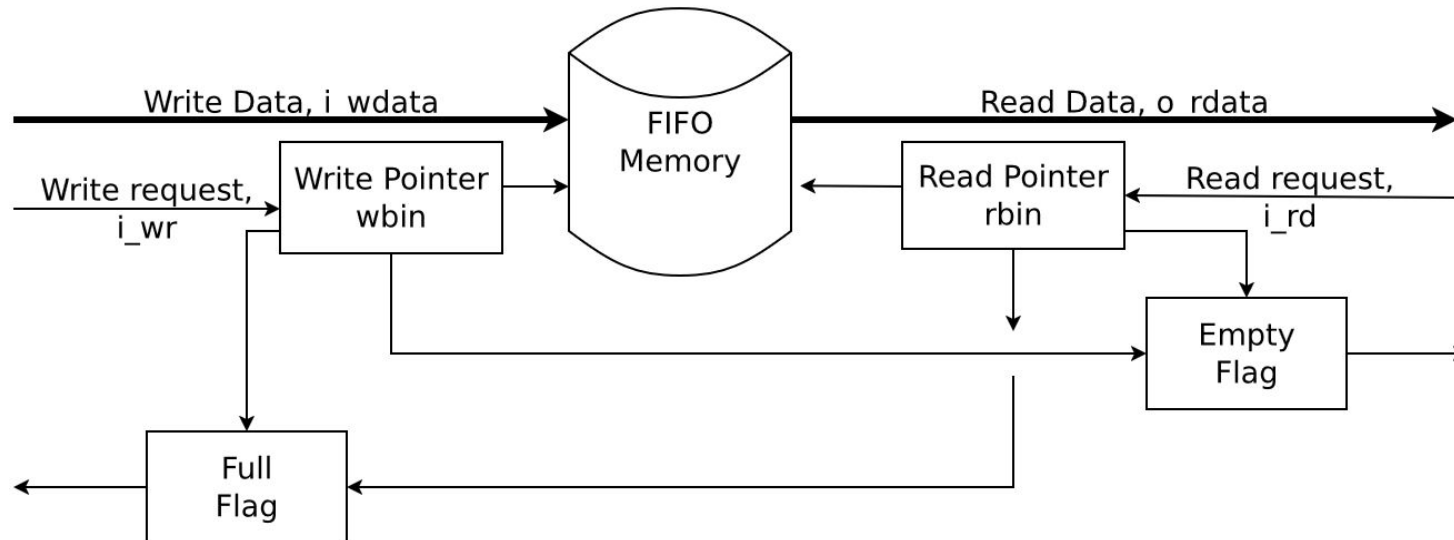




FIFO: first in first out, effectively a queue

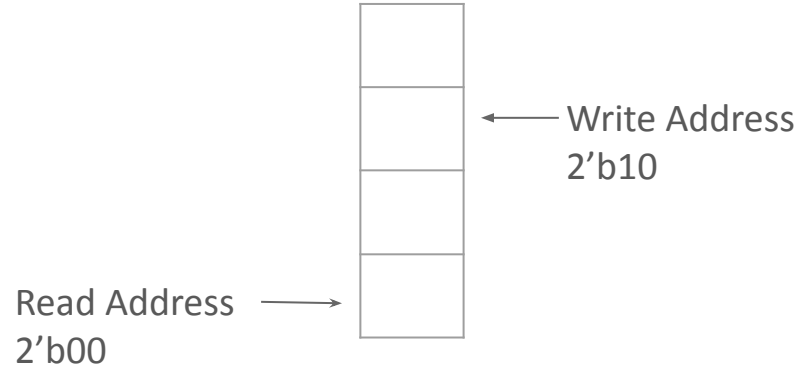
The Basic Fifo

- Centered around a 2-port memory
 - 1 read port, 1 write port
 - Could be built out of flip flops
- Write side writes at and increments write pointer
- Read side reads at and increments read pointer



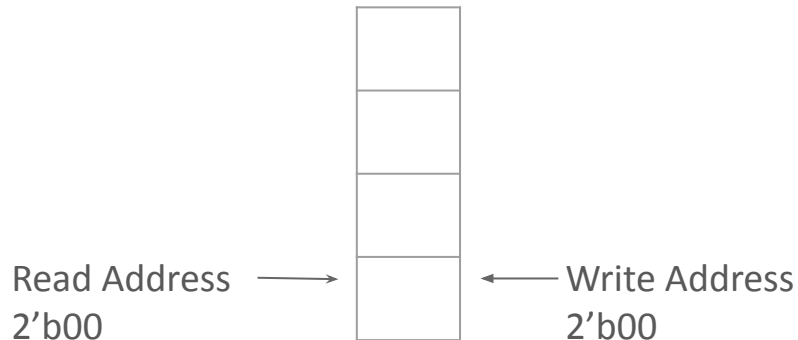
Queue Space from Addresses

- Read and Write pointers tell us how much of our queue is utilized or free
 - Read address:
 - 2 behind write address
 - has two items to read
 - Write Address
 - 2 behind read address
 - has two slots to write to



Full or Empty?

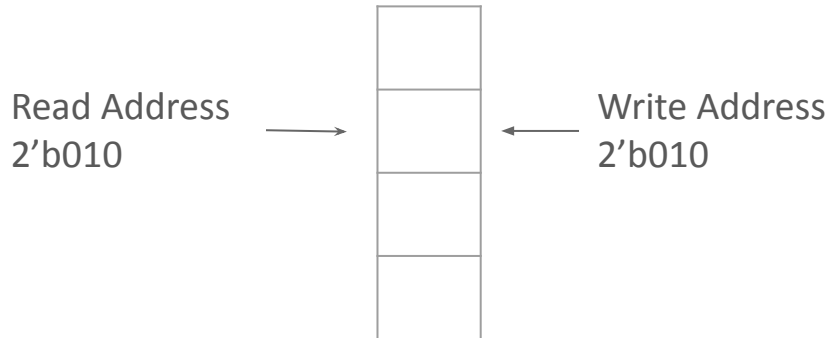
- When Queue is empty, addresses start out equal (0)
- If write pointer fills the queue, its n-bit address with roll over and be 0 again
- How can we tell if queue is empty or full from only the pointers?



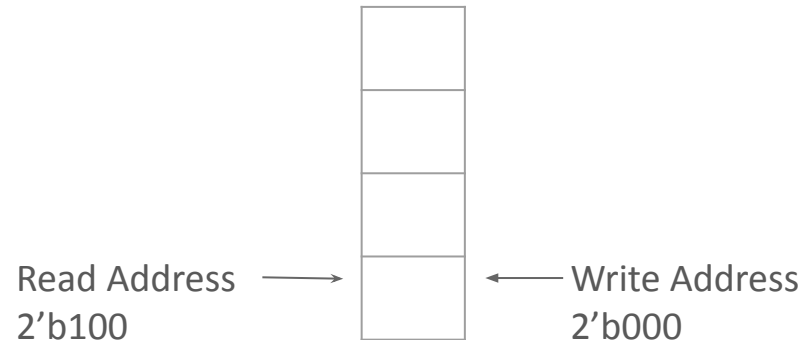
Extra Address Bit

- Use one extra bit in address to keep track of rollover
 - If addresses are the same, queue is empty
 - If top bit is different but the rest is the same, queue is full

Empty Case

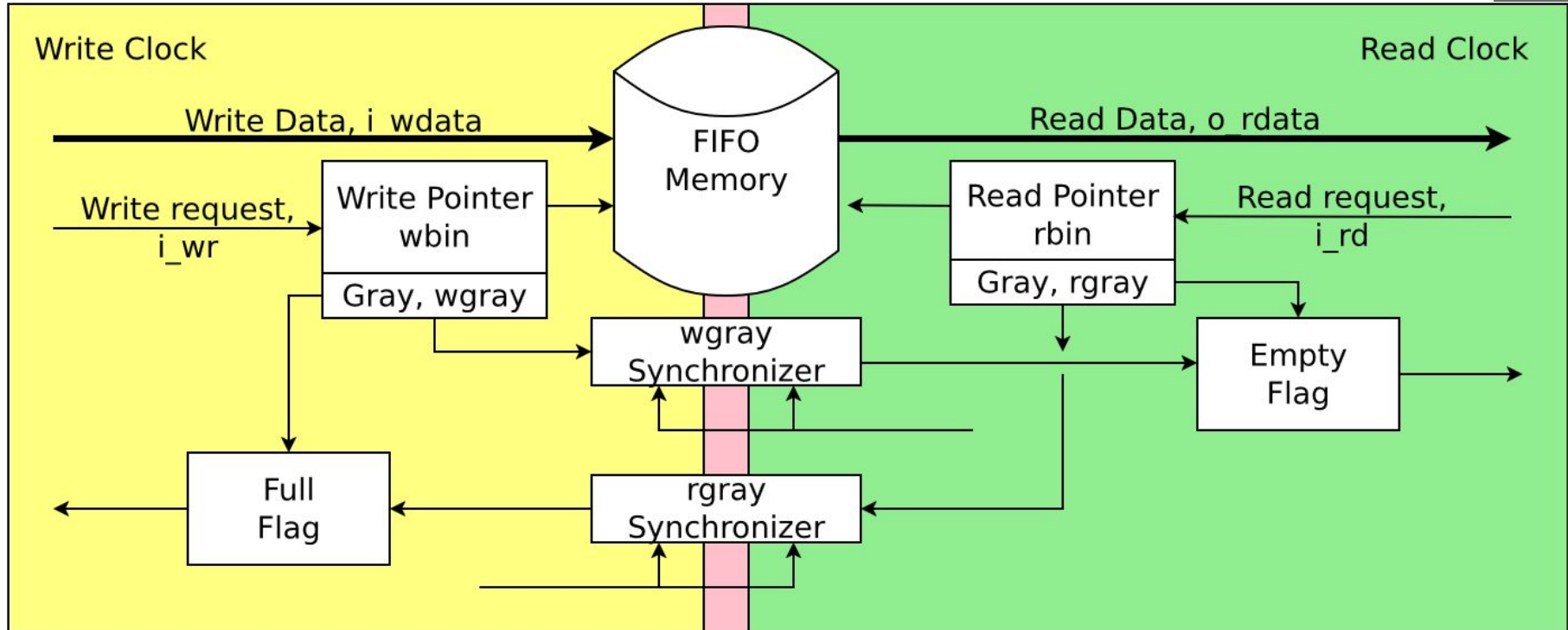


Full Case



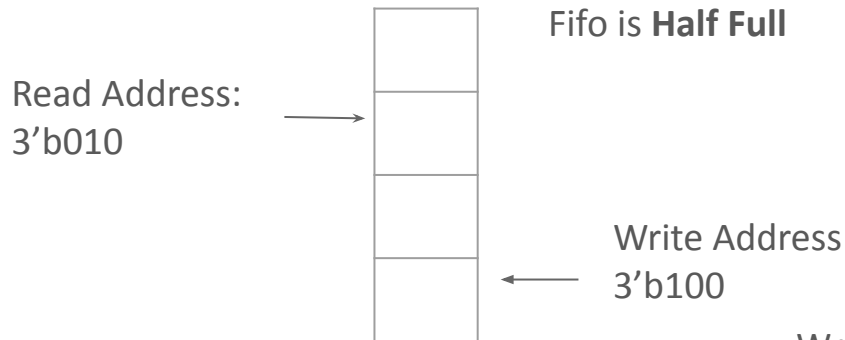
Async Fifo

- Same structure as Sync Fifo
- Use 2 Synchronizers or CDCs to pass gray-encoded read and write pointers

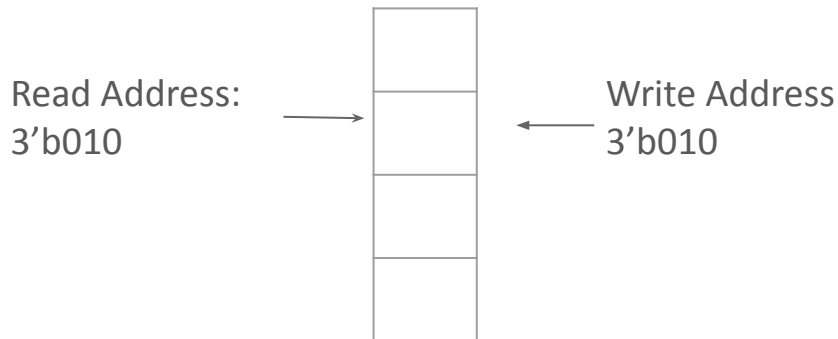


Async Fifo - Pointer Latency

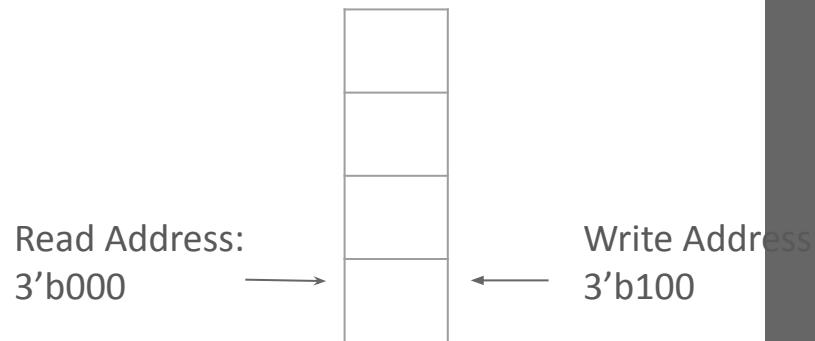
- Latency in pointers can cause slowdowns
 - Each side of fifo sees a slightly out of date view
- Cannot cause failure, only delays



Read Side POV: Fifo is **Empty**

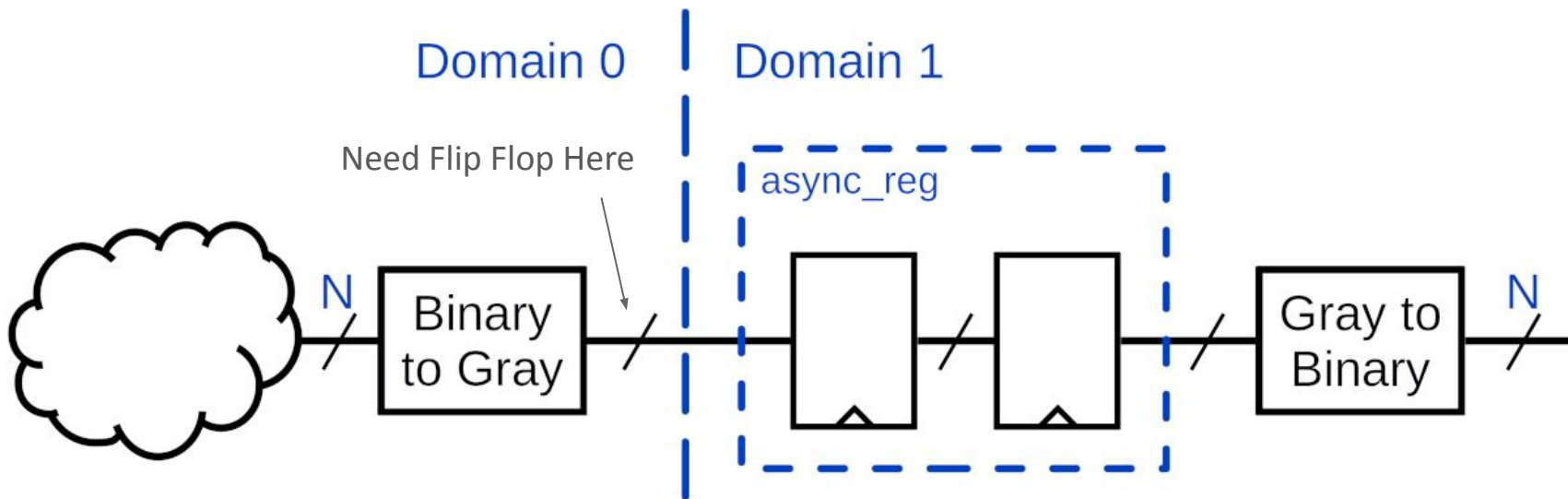


Write Side POV: Fifo is **Full**



Common Issues

- Need our gray-encoded vector to arrive synchronized across clock domains
 - Should use a flip flop after converting to gray
- Bad Idea: send gray encoded vector straight from combinational logic
 - Each bit will take a different amount of time to go through converter
 - Loss of synchronization

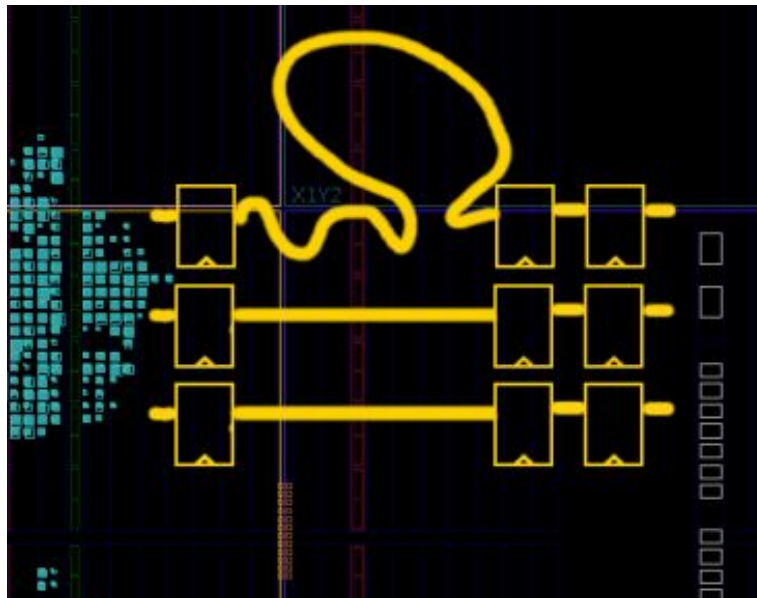


Design Constraints: Clocking

- Once we move past 1 clock, need to use SDC files to define clocks
- Use **get_ports** to interact with top-level module IO
- Use **create_clock** to tell STA tools which pins are clocks
 - Set period of each clock
- Create an Asynchronous Clock Group
 - Use **get_clocks** to interact with created clocks
 - Read and Write clocks are not synchronized
 - Use **set_clock_groups -asynchronous** to tell tools that

```
create_clock [get_ports clk_r] -name clk_r -period 7
set_propagated_clock clk_r
create_clock [get_ports clk_w] -name clk_w -period 14
set_propagated_clock clk_w
set_clock_groups -asynchronous -group [get_clocks {clk_r clk_w}]
```

Design Constraints: Max Delay



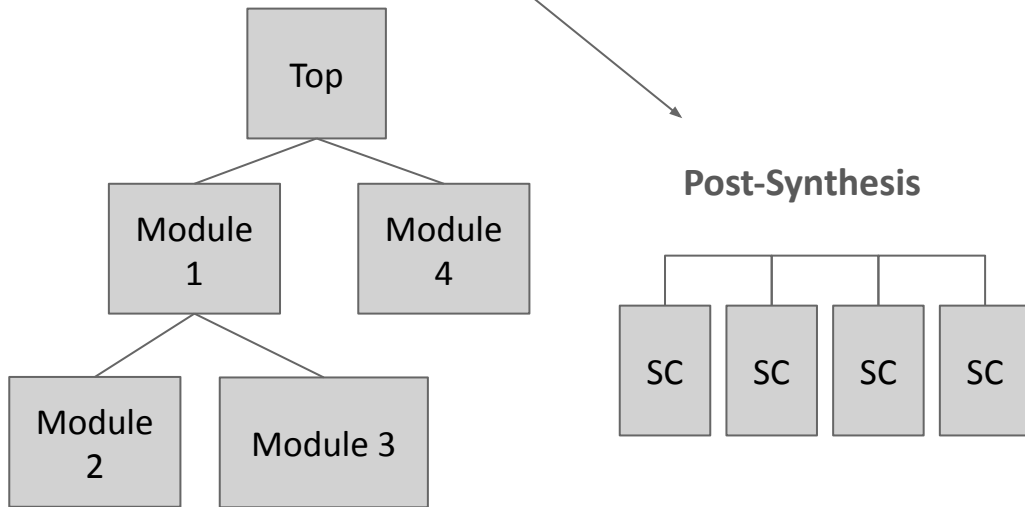
- Gray Encoded Counters rely on only one bit transitioning at a time
 - If each bit is routed extremely differently, could get un-synchronized
 - If Propagation Delay $\gg$ Min Clock Period, 2nd bit transition while first is in flight
- Tools cannot guess the max propagation between different clock domains
 - Have to tell the tools manually
 - Set max delay between clock domains = min clock period

Problem: Design Flattening

Pre-Synthesis

Design Flattening

Post-Synthesis



Problem:

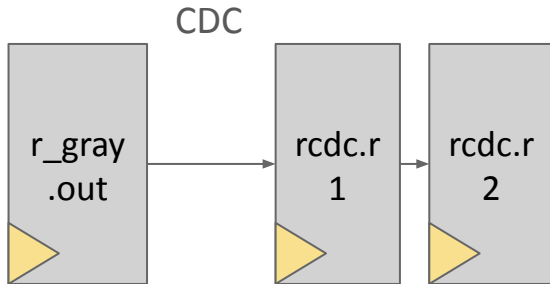
- In industry, design constraints interact with hierarchy
 - Top.Module1.Module2/Pin
 - Xilinx, Synopsys, etc.
- Open Source: OpenSTA is done after Synthesis (Yosys)
 - Design constraints need to interact with gate-level netlist
 - Have to interact with individual standard cells

Design Constraint Example

```
sky130_fd_sc_hd__dfxtp_2 \r_gray.out[0]_sky130_fd_sc_hd__dfxtp_2_Q (
  .CLK(clk_r),
  .D(\r_ptr_next[1] ),
  .Q(\r_gray.out[0] )
);|
sky130_fd_sc_hd__dfxtp_2 \r_gray.out[1]_sky130_fd_sc_hd__dfxtp_2_Q (
  .CLK(clk_r),
  .D(\r_gray.out[1]_sky130_fd_sc_hd__dfxtp_2_Q_D ),
  .Q(\r_gray.out[1] )
);
```

Want to interact with
out vector of
r_gray module

Netlist splits it into different
flip flops named **r_gray.out**
including a suffix to identify
as a flip flop

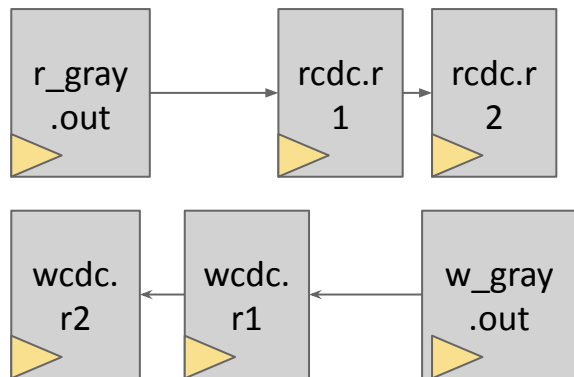


```
[get_pins r_gray.out*df*/CLK]
```

Use wildcard * to match any standard cell that:

- starts with **r_gray.out**
- contains **df** (is a d flip flop)
- has a clock output

Final Design Constraints



- Have to constrain max delay in both directions
 - Read to Write CDC
 - Write to Read CDC
- Set max delay to minimum of the two clock periods
 - Can hard code this → Could forget to change later
 - Use TCL scripting to automatically pick min period

```
set read_period [get_property -object_type clock [get_clocks {clk_r}] period]
set write_period [get_property -object_type clock [get_clocks {clk_w}] period]
set min_period [expr {min(${read_period}, ${write_period})}]
```

```
set_max_delay -from [get_pins r_gray.out*df*/CLK] -to [get_pins rcdc.r1*df*/D] $min_period
set_max_delay -from [get_pins w_gray.out*df*/CLK] -to [get_pins wcdc.r1*df*/D] $min_period
```

References

- <https://zipcpu.com/blog/2018/07/06/afifo.html>
- <https://vlsiverify.com/verilog/verilog-codes/asynchronous-fifo/>
- <https://www.intel.com/content/www/us/en/docs/programmable/683353/21-3/clock-domain-crossing-constraint-guidelines.html>
- <https://www.linkedin.com/pulse/reliable-cdc-constraints-2-counters-fifos-lukas-vik-ist5c/>